

Informe del Proyecto Minix

Sistema Operativo

Facultad de Matemática y Computación

Universidad de La Habana

Integrante: Kamila Reinoso Asin CC211

1. Introducción

El presente proyecto se desarrolla en el marco de la asignatura Sistemas Operativos de la Facultad de Matemática y Computación de la Universidad de La Habana, con el fin de profundizar en los pilares que sustentan la computación moderna. A lo largo del trabajo, se abordan principios fundamentales como la abstracción de procesos —entendidos como entidades activas con estado dinámico—, la concurrencia y sincronización de hilos, la planificación de ejecuciones para la gestión eficiente del tiempo de CPU y la interacción con el sistema de archivos. Se utiliza MINIX como plataforma de estudio por su diseño modular y pedagógico, el cual permite intervenir directamente en componentes reales del núcleo y las bibliotecas del sistema, facilitando la transición de los conceptos teóricos de la multiprogramación a la implementación práctica.

El objetivo general de este proyecto es consolidar el aprendizaje mediante la modificación y el análisis interno de un sistema operativo funcional, promoviendo la capacidad de diagnosticar fallos y proponer soluciones a nivel de sistema. Para ello, el desarrollo se estructura en cuatro hitos específicos: la personalización del entorno de usuario mediante el mensaje de bienvenida; la depuración de un error crítico en la biblioteca de hilos en el método `pthread_mutex_trylock` para garantizar la exclusión mutua; la implementación del comando `tree`, que requiere el uso de llamadas al sistema para la navegación recursiva de directorios; y, finalmente, la modificación del scheduler. En este último punto, se implementa una política de penalización para procesos intensivos en CPU, basada en ventanas temporales, con el fin de optimizar la respuesta del sistema ante tareas interactivas

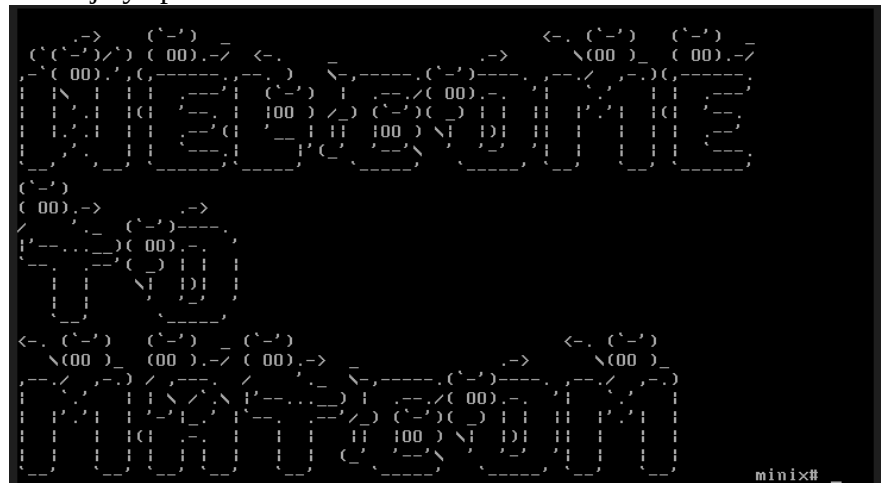
2. Desarrollo y resultados por componente

2.1 Instalación y configuración de VirtualBox y Minix

Se utilizó Oracle VirtualBox versión 7.2.6. La máquina virtual se configuró con 2 GB de RAM y 3 núcleos de CPU, superando el mínimo recomendado (1 GB RAM / 1 núcleo) para garantizar la fluidez durante la compilación del kernel y las pruebas de concurrencia. Se asignaron 40 GB de disco para prever el crecimiento de los logs de depuración y el almacenamiento de múltiples versiones de los binarios modificados. Se utilizó un adaptador NAT para permitir la descarga de herramientas y la sincronización con el repositorio de GitHub mediante git

2.2 Personalización del Mensaje de Bienvenida

Para poder editar el mensaje de bienvenida de MINIX en el archivo motd usamos la página ASCII ART GENERATOR, con el mensaje WELCOME TO MATCOM, copiado el mensaje de bienvenida en la dirección etc/motd, dado que el proyecto se ha desarrollado en el sistema operativo Ubuntu y probado su funcionamiento en Minix hemos subido este mensaje a github, descargado en Minix mediante el comando `git -c http.sslVerify=false pull origin Hito-1` dado que al hacer solo `git pull origin Hito -1` nos ha dado un error en el certificado SSL solucionado con el comando anterior. Se usa el comando `cp etc/motd /etc/motd` para copiar el archivo descargado desde GitHub en el sistema de minix a continuación se aporta una imagen de dicho mensaje ya puesto en el sistema.



2.3 Depuración del bug en pthread

El fallo en la biblioteca de hilos de MINIX es un error de diseño en la capa de compatibilidad que impide el funcionamiento correcto de la exclusión mutua cuando se utiliza el estándar POSIX.

1. Síntomas del fallo

El problema se manifiesta cuando un programa intenta realizar llamadas sucesivas a la función `pthread_mutex_trylock` sobre un mismo mutex. En un comportamiento normal, la primera llamada debería bloquear el mutex y la segunda debería retornar inmediatamente con un código de error (como `EDEADLK` o `EBUSY`) indicando que el mutex ya está ocupado. Sin embargo, en la versión con el bug, el programa se bloquea indefinidamente en la segunda llamada, quedando "colgado" antes de poder ejecutar las operaciones de desbloqueo (`unlock`) o destrucción (`destroy`).

2. Análisis de la causa raíz

La arquitectura de hilos en MINIX divide las funciones en dos niveles:

Capa nativa (mthread_*): Implementada en libmthread, es la lógica real que gestiona los mutexes en el sistema.

Capa de compatibilidad (pthread_*): Se encuentra en el archivo minix/lib/libmthread/pthread_compat.c y su función es simplemente "traducir" las llamadas de estándar POSIX a las funciones nativas de MINIX.

El error específico: Dentro de pthread_compat.c, la función pthread_mutex_trylock contenía una recursión infinita. En lugar de invocar a la función nativa del sistema (mthread_mutex_try_lock), el código se llamaba a sí mismo repetidamente. Esta sucesión de llamadas circulares agota el espacio de la pila (stack overflow), provocando que el proceso se bloquee y no pueda retornar nunca el control al programa de usuario.

3. Corrección técnica

La solución consiste en romper la recursión redirigiendo la llamada a la función correcta de la capa nativa. En el archivo pthread_compat.c, se modificó el retorno de la siguiente manera:

```
--- a/libmthread/pthread_compat.c
+++ b/libmthread/pthread_compat.c
- return pthread_mutex_try_lock(mutex);
+ return mthread_mutex_try_lock(mutex);
```

4. Verificación de la solución

Una vez aplicada la corrección, el programa de prueba arroja los siguientes resultados:



```
minix# cd /usr/src/minix/tests
minix# ls -F
Makefile      dev/          ipf/          net/          test_hito2_bug/
Makefile.inc  fs/          kernel/       rump/         test_hito3/
README        games/       lib/          sbin/         usr.bin/
bin/          h_macros.h  libexec/     share/        usr.sbin/
crypto/       include/    modules/     sys/
minix# cd test_hito2_bug
minix# ls -F
program_test* test_bug.c
minix# ./program_test
first trylock: 0 (OK)
second trylock: 11 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS
minix#
```

2.4 Implementación del comando tree

Diseño del algoritmo Para la implementación de este comando, se optó por un enfoque de recursión explícita. La función principal, tree(char *dir, int level) , se encarga de abrir un directorio, procesar sus entradas y, en caso de encontrar un subdirectorio, llamarse a sí

misma aumentando el nivel de profundidad. Esta solución es ideal para estructuras jerárquicas como los sistemas de archivos, ya que permite recorrer niveles arbitrarios de profundidad de forma natural. Llamadas al sistema utilizadas Se emplearon las siguientes llamadas al sistema estándar de C/POSIX para interactuar con el sistema de archivos de MINIX:

- `readdir`: Lee secuencialmente cada entrada (archivo o carpeta) dentro del directorio abierto.
- `lstat`: Obtiene la información del estado de la entrada. Se prefirió sobre `stat` para cumplir con el requisito de no seguir enlaces simbólicos y así evitar ciclos infinitos.
- `closedir`: Cierra el flujo del directorio para liberar los descriptores de archivos del sistema.
- `opendir`: Abre el flujo del directorio especificado en la ruta.

Indentación y visualización de profundidad La profundidad se gestiona mediante el parámetro entero `level`. Por cada nivel de profundidad alcanzado en la recursión, el programa ejecuta un ciclo `for` que imprime espacios en blanco antes del nombre de la entrada. Esto genera un efecto visual de jerarquía escalonada, donde los archivos y carpetas hijos aparecen alineados a la derecha respecto a su directorio padre.

Prevención de ciclos. Para evitar el seguimiento recursivo de enlaces simbólicos que podrían causar ciclos infinitos, el programa utiliza la función `lstat`. A diferencia de `stat`, que sigue el enlace hasta el archivo u objeto original, `lstat` obtiene la información del enlace mismo. El código verifica si la entrada es un directorio mediante la macro `S_ISDIR (info st mode)` antes de intentar una nueva llamada recursiva, asegurando que solo se exploren directorios reales y no accesos directos que apunten a otras partes del sistema.

Lógica principal y manejo de argumentos

- **Filtrado de directorios especiales:** El algoritmo incluye una cláusula `if` para ignorar las entradas. (`directorio actual`) y `..` (`directorio padre`), evitando que la recursión regrese a niveles superiores o se estanque en el mismo nivel.
- **Punto de entrada (main):** El programa permite especificar una ruta como argumento (`argv`). Si no se proporciona ninguna, el sistema asume por defecto el directorio actual.
- **Construcción de rutas:** Se utiliza `snprintf` para concatenar la ruta base con el nombre de la entrada actual, creando una ruta absoluta o relativa válida para la siguiente llamada recursiva

Ejemplo con comando tree /usr

```
minix# tree /usr
```

```
gtest-typed-test.cc
gtest.cc
update_llc_test_checks.py
valgrind
i386-pc-linux-gnu.supp
x86_64-pc-linux-gnu.supp
vim
README
llvm.vim
tablegen.vim
vimrc
wcii.py
yaml-bench
CMakeLists.txt
Makefile
YAMLBench.cpp
include
Makefile
lib
Makefile
Makefile.inc
libLLVMMArch64AsmParser
Makefile
libLLVMMArch64AsmPrinter
```

Ejemplo con la ruta /usr/src/minix/minix/servers/sched

```
minix# tree /usr/src/minix/minix/servers/sched
Makefile
main.c
proto.h
sched.h
schedproc.h
schedule.c
utility.c
gdbinit
main.o
schedule.o
utility.o
sched
minix# -
```

Ejemplo con la ruta actual

```
minix# cd /usr/src/minix/tests
minix# tree
```

```
d_merge_CMS.out
mtree_d_create.out
meths6_d_create.out
mtree.sh
tcpdump
Makefile
t_tcpdump.sh
traceroute
Makefile
t_traceroute.sh
useradd
Makefile
t_useradd.sh
test_hito2_bug
test_bug.c
program_test
test_hito3
CPU_bound.c
Intensivo.c
Mixto.c
a.out
cpu_bound
Intensivo
mixto
minix#
```

2.5 Penalización por uso intensivo de CPU

En esta parte se comenzará a hablar acerca de las modificaciones realizadas en el planificador de MINIX para implementar un mecanismo sencillo de ajuste de prioridad para un uso sostenido de CPU en una ventana de tiempo. En la dirección de minix/servers/sched es donde se gestiona la planificación de los procesos que se ejecutan dentro de este sistema operativo, específicamente en el programa Schedule.c se encuentra el método `do_noquantum` especializado en bajar la prioridad a aquellos

procesos que han utilizado su quantum completo dentro de un intervalo de tiempo. Dentro de esta carpeta se encuentra también el schedproc.h en donde se encuentra el struct con la información de cada proceso como su prioridad, máxima prioridad, la CPU actual que lo ejecuta, quantum, entre otros.

En 5 segundos se pueden ejecutar 25 penalizaciones en caso de que no se le ponga un límite a este, al hacer esto cada proceso puede gastar por completo 10 veces su tiempo llevado por un contador, completado esto se considerara como un proceso que consume mucha CPU bajándole la prioridad. Cada 5 segundos para los procesos que no han gastado nunca su quantum completo se consideran procesos intensivos subiéndole gradualmente su prioridad en el método de balance_queues.

A continuación, se muestra el código modificado de estos métodos:

```
EXTERN struct schedproc
{
    endpoint_t endpoint; /* id de endpoint del proceso */
    endpoint_t parent; /* id de endpoint del padre */
    unsigned flags; /* bits de banderas */

    /* Planificación en espacio de usuario */
    unsigned max_priority; /* la prioridad máxima permitida de este proceso */
    unsigned priority; /* la prioridad actual del proceso */
    unsigned time_slice; /* el quantum de tiempo de este proceso */
    unsigned count_quantums; /* el número de quantums que el proceso ha consumido */
    unsigned cpu; /* en qué CPU se está ejecutando el proceso */
    bitchunk_t cpu_mask[BITMAP_CHUNKS(CONFIG_MAX_CPUS)]; /* en qué CPUs tiene permitido ejecutarse el proceso */
} schedproc[NR_PROCS];

/* Flag values */
#define IN_USE 0x00001 /* set when 'schedproc' slot in use */

/*-----*
 * do_noquantum
 *-----*/
int do_noquantum(message *m_ptr)
{
    register struct schedproc *rmp;
    int rv, proc_nr_n;

    if (sched_isokendpt(m_ptr->m_source, &proc_nr_n) != OK)
    {
        printf("SCHED: WARNING: got an invalid endpoint in 000 msg %u.\n",
            m_ptr->m_source);
        return EBADEPT;
    }

    rmp = &schedproc[proc_nr_n];
    // Contador de quantums gastados por el proceso
    rmp->count_quantums++;
    printf("SCHED: penalizando pid=%d nueva priority=%d\n", proc_nr_n, rmp->priority);
    /* Si el proceso se ha quedado sin quantum demasiadas veces, bajamos su prioridad */
    if (rmp->priority < MIN_USER_Q && rmp->count_quantums >= LIMIT)
    {
        rmp->priority -= 1; /* lower priority */
    }

    if ((rv = schedule_process_local(rmp)) != OK)
    {
        return rv;
    }
    return OK;
}
```

Se añade un contador (count_quantums) para llevar un conteo de cuantos quantums ha gastado un proceso para poder ser penalizado.

En este método es donde se gestiona la penalización de cada proceso una vez gastado su límite de quantums. Una vez obtenido el proceso y comprobado su validez se le suma 1 al contador de este y se verifica que su contador ha superado el límite además de que su prioridad no sea inferior a la mínima admisible por el sistema. Una vez esto se guardan los cambios y se continua.

```

/*=====
 * balance_queues
 *=====*/

/* This function is called every N ticks to rebalance the queues. The current
 * scheduler bumps processes down one priority when ever they run out of
 * quantum. This function will find all processes that have been bumped down,
 * and pulls them back up. This default policy will soon be changed.
 */
void balance_queues(void)
{
    struct schedproc *rmp;
    int r, proc_nr;

    // Subirle la prioridad solo a los que no han gastado ningun quantum
    for (proc_nr = 0, rmp = schedproc; proc_nr < NR_PROCS; proc_nr++, rmp++)
    {
        if (rmp->flags & IN_USE)
        {
            printf("SCHED: balanceando pid=%d priority=%d\n, max=%d count=%d\n", proc_nr, rmp->priority,
                    rmp->max_priority, rmp->count_quantums);
            if (rmp->priority > rmp->max_priority && rmp->count_quantums == 0)
            {
                rmp->priority -= 1; /* increase priority */
                schedule_process_local(rmp);
            }
            rmp->count_quantums = 0; /* reset quantum count for the next round */
        }
    }

    if ((r = sys_setalarm(balance_timeout, 0)) != OK)
        panic("sys_setalarm failed: %d", r);
}

```

En la función de Balance_queues es en donde se analiza si subirle la prioridad a los procesos, en donde la condición es si dichos procesos no han gastado nunca su quantum en dicho caso se considerara un proceso intensivo y es apto para subirle su prioridad gradualmente, se comprueba que el proceso esté siendo usado, luego de esto se

verifica de que su prioridad sea mayor que su prioridad máxima para saber que una vez subida su prioridad no se pasara del límite, además de esto se mira si ha gastado algún quantum una vez verificada estas condiciones es un proceso apto y por lo cual es posible subirle la prioridad, una vez esto se guarda su prioridad y se reinicia su contador.

Para poder comprobar la validez de estas modificaciones se han creado los programas de CPU_bound que es un ciclo infinito sin ningún bloqueante, el programa Intensivo que es un ciclo while que se bloquea durante 2 segundos en cada iteración, y también se encuentra el programa Mixto que al igual que los demás realiza un ciclo while con un for que itera desde hasta 100000000000 y luego de esto otro ciclo for desde 0 hasta 40 que se bloquea durante 2 segundos por cada iteración.

```

load averages: 1.03, 0.26, 0.08; up 0:01:00:49 15:40:30
48 processes: 2 runnable, 46 sleeping
CPU states: 100% user, 0.0% nice, 0.0% system, 0.0% kernel, 0.0% idle
Memory: 2047M Total, 1940M Free, 1659M Contig, 83M Cached

PID USERNAME PRI NICE SIZE RES STATE TIME WCPU CPU COMMAND
699 root 15 0 1224K 260K RUN 1:14 95.65% 93.41% cpu_bound
702 root 7 0 1240K 396K select 0:01 0.98% 0.83% mixto
33 service 7 0 204K 204K (any) 0:30 0.00% 0.00% at_wini
9 root 1 0 212K 192K (any) 0:28 0.00% 0.00% tty
7 root 5 0 1516K 1200K (any) 0:02 0.00% 0.00% vfs
141 service 5 0 60M 60M (any) 0:01 0.00% 0.00% mfs
53 service 5 0 6384K 6384K (any) 0:01 0.00% 0.00% mfs
11 root 2 0 8536K 8536K (any) 0:01 0.00% 0.00% vm
10 root 7 0 432K 432K RUN 0:00 0.00% 0.00% mib
298 service 7 0 1304K 1304K (any) 0:00 0.00% 0.00% lwip
96 root 7 0 1168K 1168K (any) 0:00 0.00% 0.00% is
500 root 7 0 2236K 844K select 0:00 0.00% 0.00% syslogd
500 root 7 0 896K 656K select 0:00 0.00% 0.00% dhcpcd
683 root 7 0 824K 584K wait 0:00 0.00% 0.00% sh
41 root 7 0 568K 564K (any) 0:00 0.00% 0.00% procfs
705 root 7 0 736K 500K sysctl 0:00 0.00% 0.00% top
1 root 7 0 420K 420K wait 0:00 0.00% 0.00% init
704 root 7 0 1224K 372K select 0:00 0.00% 0.00% intensivo
314 root 7 0 476K 288K (any) 0:00 0.00% 0.00% uds

```

En esta imagen se puede ver como se ejecutan los 3 procesos en la tabla de procesos, se observa el como el proceso cpu_bound se mantiene en la prioridad 15 dado que nunca sube por su uso intenso de CPU, se encuentra también el proceso mixto, del que hemos hablado antes, este comienza en 12 y ha ido subiendo hasta la máxima prioridad para los procesos de usuario gracias a sus estados bloqueantes, también esta el proceso intensivo dicho antes como un proceso bloqueante por lo que su prioridad es la máxima posible. Se prueba a continuación el estado del proceso mixto.

```

load averages: 1.05, 0.42, 0.14; up 0:01:09:34 15:41:15
48 processes: 3 runnable, 45 sleeping
CPU states: 98.6% user, 0.0% nice, 0.3% system, 1.0% kernel, 0.0% idle
Memory: 2047M Total, 1940M Free, 1659M Contig, 83M Cached

PID USERNAME PRI NICE SIZE RES STATE TIME WCPU CPU COMMAND
699 root 15 0 1224K 260K RUN 1:57 87.81% 87.60% cpu_bound
702 root 15 0 1240K 396K RUN 0:03 8.33% 8.20% mixto
33 service 7 0 204K 204K (any) 0:30 0.00% 0.00% at_wini
9 root 1 0 212K 192K (any) 0:28 0.00% 0.00% tty
7 root 5 0 1516K 1200K (any) 0:02 0.00% 0.00% vfs
141 service 5 0 60M 60M (any) 0:01 0.00% 0.00% mfs
53 service 5 0 6384K 6384K (any) 0:01 0.00% 0.00% mfs
11 root 2 0 8536K 8536K (any) 0:01 0.00% 0.00% vm
10 root 7 0 432K 432K RUN 0:00 0.00% 0.00% mib
298 service 7 0 1304K 1304K (any) 0:00 0.00% 0.00% lwip
96 root 7 0 1168K 1168K (any) 0:00 0.00% 0.00% is
500 root 7 0 2236K 844K select 0:00 0.00% 0.00% syslogd
500 root 7 0 896K 656K select 0:00 0.00% 0.00% dhcpcd
683 root 7 0 824K 584K wait 0:00 0.00% 0.00% sh
41 root 7 0 568K 564K (any) 0:00 0.00% 0.00% procfs
705 root 7 0 736K 500K sysctl 0:00 0.00% 0.00% top
1 root 7 0 420K 420K wait 0:00 0.00% 0.00% init
704 root 7 0 1224K 372K select 0:00 0.00% 0.00% intensivo
314 root 7 0 476K 288K (any) 0:00 0.00% 0.00% uds

```

```

load averages: 1.11, 0.49, 0.16; up 0+01:09:49 15:41:30
48 processes: 2 runnable, 46 sleeping
CPU states: 98.3% user, 0.0% nice, 0.3% system, 1.4% kernel, 0.0% idle
Memory: 2047M Total, 1940M Free, 1659M Contig, 83M Cached

  PID USERNAME PRI NICE  SIZE  RES STATE   TIME  WCPU  CPU COMMAND
699 root      15  0 1224K 260K RUN    2:10 86.67% 86.57% cpu_bound
702 root      13  0 1240K 396K select 0:04 0.81% 0.74% mixto
  0 root       0  0 3268K 3268K kernel 0:00 0.05% 0.05% [kernel]
33 service   7  0 204K 204K (any) 0:39 0.00% 0.00% at_wini
  9 root       1  0 212K 192K (any) 0:28 0.00% 0.00% tty
  7 root       5  0 1516K 4280K (any) 0:02 0.00% 0.00% vfs
141 service  5  0 60M 60M (any) 0:01 0.00% 0.00% mfs
53 service   5  0 6384K 6384K (any) 0:01 0.00% 0.00% mfs
11 root       2  0 8536K 8536K (any) 0:01 0.00% 0.00% vm
10 root       7  0 432K 432K RUN    0:00 0.00% 0.00% mib
298 service  7  0 1304K 1304K (any) 0:00 0.00% 0.00% lwip
  96 root      7  0 1168K 1168K (any) 0:00 0.00% 0.00% is
588 root      7  0 2236K 844K select 0:00 0.00% 0.00% syslogd
580 root      7  0 896K 656K select 0:00 0.00% 0.00% dhcpcd
683 root      7  0 824K 584K wait   0:00 0.00% 0.00% sh
41 root       7  0 568K 564K (any) 0:00 0.00% 0.00% procs
705 root      7  0 736K 500K sysctl 0:00 0.00% 0.00% top
  1 root       7  0 428K 428K wait   0:00 0.00% 0.00% init
704 root      7  0 1224K 372K select 0:00 0.00% 0.00% intensio

```

3.Resultados globales y discusión

Tras la culminación de las fases de desarrollo, se confirma que el proyecto THE MATCOM MINIX ha sido completado satisfactoriamente, cumpliendo con el 100 % de los criterios de aceptación establecidos en la orientación original. El estado final de los componentes es el siguiente:

- Instalación y Personalización: El entorno VirtualBox/MINIX es plenamente operativo y el archivo /etc/motd refleja correctamente la identidad de la facultad.
- Depuración de pthread: Se resolvió el fallo en pthread_mutex_trylock, permitiendo que el programa de prueba supere todas las validaciones (PASS) sin quedar bloqueado en llamadas sucesivas.
- Comando tree: La herramienta recorre de forma recursiva cualquier ruta proporcionada, gestionando la profundidad mediante indentación visual y evitando ciclos infinitos mediante el uso de lstat.
- Modificación del Scheduler: Se implementó con éxito la política de penalización en el servicio sched, logrando que el sistema identifique y reduzca la prioridad de procesos CPU-bound de forma dinámica.

Análisis y Lecciones Aprendidas

La discusión técnica de este proyecto gira en torno a la transición de un sistema de planificación estático a uno dinámico. Una de las lecciones más importantes sobre el funcionamiento interno de MINIX es que, por defecto, el sistema tiende a tratar a los procesos de usuario con un esquema de prioridades similares o estáticas dentro de sus colas. Al intervenir el planificador, se comprendió que mantener una equidad absoluta (ejecutar procesos con igual prioridad) puede ser ineficiente cuando coexisten tareas interactivas y tareas intensivas en cómputo. La implementación de la penalización por agotamiento de quantum permitió observar cómo el sistema operativo puede "aprender" del comportamiento del proceso en ventanas temporales para favorecer la interactividad, un principio fundamental de los algoritmos MLFQ estudiados en clase.

En cuanto a la concurrencia, la depuración de la biblioteca de hilos evidenció que incluso errores mínimos en la gestión del estado de un mutex pueden derivar en condiciones de interbloqueo (*deadlock*) que inhabilitan la ejecución de un programa. Por último, el desarrollo del comando tree reforzó la importancia del uso de llamadas al sistema POSIX (`opendir`, `readdir`, `lstat`) para la manipulación directa del sistema de archivos. El uso de `lstat` resultó crítico: se aprendió que la abstracción de archivos en UNIX/MINIX incluye punteros (enlaces simbólicos) que, de no ser manejados correctamente mediante la obtención del estado del enlace y no del destino, pueden comprometer la estabilidad del algoritmo recursivo. En conclusión, los resultados obtenidos son plenamente coherentes con los objetivos de aprendizaje práctico,

demostrando una integración efectiva entre la teoría de sistemas operativos y la implementación real.

4. Conclusiones

Tras la ejecución de este proyecto, se concluye que los objetivos generales han sido cumplidos satisfactoriamente, logrando consolidar el aprendizaje teórico mediante la intervención directa en el núcleo y las bibliotecas de MINIX. La práctica permitió una comprensión profunda de cuatro pilares fundamentales de los sistemas operativos: la personalización del entorno, la sincronización de hilos, la navegación del sistema de archivos y la planificación de procesos.

En cuanto a los componentes específicos, se alcanzó un 100 % de éxito en todas las actividades:

- La abstracción de hilos y concurrencia fue comprendida al diagnosticar y corregir el error de recursión infinita en `pthread_mutex_trylock`, lo que reforzó la importancia de diferenciar entre capas de compatibilidad y funciones nativas para evitar estados de interbloqueo o fallos críticos de memoria.
- La gestión del sistema de archivos se aplicó mediante el comando `tree`, donde el uso de llamadas al sistema como `opendir`, `readdir` y, especialmente, `lstat`, demostró ser vital para el manejo seguro de la recursión y la prevención de ciclos infinitos causados por enlaces simbólicos.
- La planificación de ejecuciones se abordó mediante la transición de un esquema de prioridades estáticas a una política dinámica de penalización en el scheduler. Se aplicaron con éxito conceptos de algoritmos tipo MLFQ, permitiendo que el sistema "aprenda" del comportamiento de los procesos para favorecer la interactividad frente a tareas intensivas en CPU.

Como posibles mejoras, se sugiere extender la funcionalidad del comando `tree` para incluir caracteres gráficos de dibujo de cajas similares a los de Linux, o profundizar en el scheduler implementando un mecanismo de *priority boost* más complejo para mitigar totalmente el riesgo de inanición en procesos de baja prioridad.

En última instancia, esta experiencia ha sido fundamental para la formación en Ciencia de la Computación, ya que permitió cerrar la brecha entre los conceptos abstractos discutidos en las conferencias y la implementación técnica real en un sistema operativo funcional. La capacidad de diagnosticar fallos a nivel de kernel y proponer soluciones rigurosas constituye una competencia técnica invaluable para el desarrollo de software de sistemas.

5. Referencias consultadas

- MINIX 3 Project. (s.f.). *The MINIX 3 Operating System*. Recuperado de <http://www.minix3.org/>.
- Tanenbaum, A. S. (2014). *Modern Operating Systems* (4ta ed.). Pearson.

- Tanenbaum, A. S., & Woodhull, A. S. (2006). *Operating Systems: Design and Implementation* (3ra ed.). Prentice Hall
- Universidad de La Habana. (2026). *Sistemas Operativos: Guía de estudio. Conferencia 2: Control y planificación de ejecuciones. Procesos.*
- Universidad de La Habana. (2026). *Sistemas Operativos: Guía de estudio. Conferencia 4: Control y planificación de ejecuciones. Hilos.*
- Universidad de La Habana. (2026). *Sistemas Operativos: Guía de estudio. Conferencia 5: Planificación.*
- IEEE Computer Society. (2017). *IEEE Std 1003.1-2017 (POSIX.1-2017).* (Referencia para las funciones de hilos y sistema de archivos utilizadas: pthread, lstat, opendir)

6. Declaración de uso de IA

Partes del proyecto donde se usó IA:

- Redacción técnica del informe: Estructuración de las secciones y refinamiento del lenguaje para cumplir con el tono formal exigido por la facultad.
- Explicación de conceptos: Clarificación del funcionamiento de las llamadas al sistema (como lstat y readdir) y de los algoritmos de planificación.
- Búsqueda de información: Localización de datos específicos dentro de la documentación oficial de MINIX y los libros de texto de la asignatura.

Naturaleza del uso:

- Asistencia en la redacción: Se utilizó la herramienta para organizar las ideas y borradores iniciales en párrafos coherentes y técnicamente rigurosos, siguiendo la estructura de la plantilla oficial.
- Apoyo didáctico: La IA facilitó la comprensión profunda de cómo las llamadas al sistema gestionan la abstracción de archivos y la prevención de ciclos en estructuras recursivas.
- Investigación: Agilización en la consulta de referencias bibliográficas (como Tanenbaum) y especificaciones del estándar POSIX para asegurar la validez de la implementación técnica.

7. Anexos

- [KamAlien/minix: Official MINIX sources - Automatically replicated from gerrit.minix3.org](https://gerrit.minix3.org/KamAlien/minix)